

07Bikes-Linear-Regression

October 5, 2021

1 Data Science: Bridging Principle and Practice

1.1 Part 7: Linear Regression Model (Bike Sharing case study)

```
<div style="float:left;width:40%">
  
</div>
<div style="float:left;width:40%">
  
</div>
```

1.1.1 Table of Contents

```
<li><a href="#section7">Linear Regression Model</a>
  <ol type=a>
    <br>
    <li><a href="#section7a">Explanatory and Response Variables</a></li>
    <br>
    <li><a href="#section7b">Finding  $\beta$ </a></li>
    <br>
    <li><a href="#section7c">Evaluating the Model</a></li>
  </ol>
</li>
</ol>
```

```
In [1]: # run this cell to import some necessary packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
%matplotlib inline
import seaborn as sns
from gofer.ok import check
import warnings
warnings.filterwarnings('ignore')

import ipywidgets as widgets
```

```

from scipy.linalg import lstsq
from sklearn.model_selection import train_test_split

def make_X(df, var_names):
    """Given a DataFrame and a list of explanatory variables, one-hot encodes
    variables if they are categorical and returns a dataframe with
    all the given explanatory variables."""
    categorical = ["month", "week day", "season"]
    boolean = ["is holiday", "is work day"]
    X = pd.DataFrame({"intercept": np.ones(df.shape[0], dtype='int')}, index = df.index)
    for var in var_names:
        if var in categorical:
            dummies = pd.get_dummies(df[var])
            formatted = dummies.drop(dummies.columns[-1], axis=1)
        elif var in boolean:
            formatted = (df[var] == "yes") * 1
        else:
            formatted = df.loc[:, var]
        X = X.join(formatted)

    return X

```

1.2 Case Study: Capital Bike Share

Bike-sharing systems have become increasingly popular worldwide as environmentally-friendly solutions to traffic congestion, inadequate public transit, and the "last-mile" problem. Capital Bikeshare runs one such system in the Washington, D.C. metropolitan area.

The Capital Bikeshare system comprises docks of bikes, strategically placed across the area, that can be unlocked by *registered* users who have signed up for a monthly or yearly plan or by *casual* users who pay by the hour or day. They collect data on the number of casual and registered users per hour and per day.

Let's say that Capital Bikeshare is interested in a **prediction** problem: predicting how many riders they can expect to have on a given day. UC Irvine's [Machine Learning Repository](#) has combined the bike sharing data with information about weather conditions and holidays to try to answer this question.

In this notebook, we'll walk through the steps a data scientist would take to answer this question.

```

In [2]: # run this cell to load the data
bikes = pd.read_csv("data/day_renamed_dso.csv")

# set the random seed
np.random.seed(28)

# split the data into training and test sets
bike_train, bike_test = train_test_split(bikes, train_size=0.8, test_size=0.2)

# reformat the date column from strings to dates

```

```
bike_train['date'] = pd.to_datetime(bike_train['date'])
bike_test['date'] = pd.to_datetime(bike_test['date'])
```

1.3 7. The Regression Model

To try to predict the number of riders on a given day, we'll use a regression model.

A **simple regression model** describes how the conditional mean of a response variable y depends on an explanatory variable x :

$$\mu_{y|x} = \beta_0 + \beta_1 x$$

This equation describes our best guess for y given a particular x .

A **multiple regression model** describes how the conditional mean of a response variable y depends on multiple explanatory variables $x_1, x_2, \dots, x_k$:

$$\mu_{y|x_1, x_2, \dots, x_k} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k$$

This equation describes our best guess for y given a particular vector $x_1, x_2, \dots, x_k$.

In this case, our model will look something like this:

$$\mu_{\text{rider count}|\text{temp, humidity, ..., month}} = \beta_0 + \beta_1 * (\text{temp}) + \beta_2 * (\text{humidity}) + \dots + \beta_k * (\text{month})$$

The code for either a simple or multiple regression model is basically identical except for the number of columns we select for inclusion in our explanatory variables.

To create our model, we need to: 1. isolate the explanatory variables (X) and response variable (y) we want to use 2. find the values for the β variables on the best-fit regression line 3. evaluate the accuracy of our model

1.3.1 7a. Explanatory and response variables

First, let's decide on our response variable. We'll try to predict the *total number of riders* on a given day. The response variable needs to be in an array (not a DataFrame), so we'll get it using indexing.

```
In [3]: # response variable: total count of riders in a day (training set)
        y_train = bike_train['total riders']

        # response variable: total count of riders in a day (validation set)
        y_test = bike_test['total riders']

        # show the first 5 items in the array
        y_train.head()
```

```
Out[3]: 215    4576
        570    7592
        11     1162
        181    5362
        303    3669
        Name: total riders, dtype: int64
```

Next, we want to choose our explanatory variables. Let's try predicting ridership in terms of *temperature*, *work day*, and *season*.

Why don't we just use all the explanatory variables?

You might think that the best model would use *all* the available explanatory information

Here, we run into a problem: our DataFrame has several categorical variables, including "season" and "week day". Categorical variables seem like they might be helpful in predicting the number of riders; however, we can't feed text into a mathematical model. And, if we code them as integers, as they were coded in the original data set (e.g. Monday = 1, Tuesday = 2, ...) that can lead to questionable manipulations. For example, if Sunday is coded as 0 and Saturday is coded as 6, the computer might conclude that the average of Sunday and Saturday is Wednesday (since Wednesday is coded as 3).

One-Hot Encoding To work around this, we will **one-hot encode** all our categorical variables. In one-hot encoding, the possible values of the variable each get their own column of 1s and 0s. The value is a 1 if that day falls in that category and a 0 otherwise.

Here's an example. Say we have three possible weather states: rain, cloudy, or sunny.

```
In [4]: # an example DataFrame of weather states
categorical = pd.DataFrame({"weather": ["rainy", "cloudy", "sunny", "cloudy"]})
categorical
```

```
Out[4]:   weather
0    rainy
1   cloudy
2    sunny
3   cloudy
```

The one-hot encoding would look like this:

```
In [5]: # pd.get_dummies is a function that does dummy encoding
one_hot = pd.get_dummies(categorical['weather'])
one_hot
```

```
Out[5]:   cloudy  rainy  sunny
0         0       1       0
1         1       0       0
2         0       0       1
3         1       0       0
```

Notice that in each row, only one of the values is equal to 1 (hence the name, "one-hot" encoding), since no day can have more than one weather state.

Notice also that we don't technically need the third column.

```
In [6]: one_hot.drop("sunny", axis=1)
```

```
Out [6]:
```

	cloudy	rainy
0	0	1
1	1	0
2	0	0
3	1	0

If we know that there are only three possible weather states, and we see that day 2 was neither cloudy nor rainy (that is, cloudy=0 and rainy=0), day 2 *must* have been sunny. This is helpful to save computation time and space. If you have some knowledge of linear algebra, note that this is also helpful to solve the problem of *perfect multicollinearity*- a situation that can make it impossible to compute the optimal set of β s.

For simplicity, we've provided a function called `make_X` that will convert our data to the correct format for prediction, including one-hot encoding the categorical variables. `make_X` takes two arguments: * the first argument, `df`, is the DataFrame with the data we want to use to make predictions * the second argument, `var_names`, is a list with the names of all the explanatory variables in `df` we want our model to use to make predictions: categorical and numerical

`make_X` will also add a column called "intercept" that only contains 1s. This column will help find the intercept term β_0 in our regression line. You can think of the intercept term as an x_0 that gets multiplied by β_0 and is always equal to 1.

```
In [7]: # select the explanatory variables to use in a DataFrame
expl_vars = ["temp", "is work day", "season"]

# convert the explanatory table to the correct format
X_train = make_X(bike_train, expl_vars)

# show the first five rows of the model inputs
X_train.head()
```

```
Out [7]:
```

	intercept	temp	is work day	fall	spring	summer
215	1	0.710000	1	1	0	0
570	1	0.750833	1	1	0	0
11	1	0.172727	1	0	1	0
181	1	0.722500	1	1	0	0
303	1	0.340000	1	0	0	0

EXERCISE:

Since we want to try the model on the test data as well, we will also perform the same train

</p>

<p>Hint: we'll need to go through the exact same steps as in the above cell for the training data

</p>

```
In [8]: # select the explanatory variables to use in a DataFrame
expl_vars = ["temp", "is work day", "season"]

# convert the explanatory table to the correct format
X_test = make_X(bike_test, expl_vars)
```

```
# show the first five rows of the model inputs
X_test.head()
```

```
Out[8]:
```

	intercept	temp	is work day	fall	spring	summer
431	1	0.404167	1	0	1	0
225	1	0.676667	0	1	0	0
491	1	0.562500	0	0	0	1
231	1	0.697500	0	1	0	0
607	1	0.706667	1	1	0	0

```
In [9]: # run this cell to check your work
check("tests/expl-matrix.ok")
```

```
Out[9]: <gofer.ok.OKTestsResult at 0x7f8692d47630>
```

1.3.2 7b. Finding β

The next step is to calculate the β terms. We can do this with a function from the [Scipy statistical analysis Python package](#) called `lstsq`.

Given a matrix of explanatory variables X and an array of response variables y , `lstsq` returns a vector β . `lstsq` uses *ordinary least squares* as its **loss function (L)**: the function that defines the training loss (error) and what we seek to minimize (often using linear algebra or calculus, depending on the loss function). The ordinary least squares equation is a common loss function that is used to minimize the sum of squared errors:

$$L(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \text{predicted}(\beta, x_i))^2$$

where n is the number of days, y_i is the actual number of riders on the i th day, and $\text{predicted}(\beta, x_i)$ is number of riders predicted to be on the i th day when using β and the explanatory variables x_i . When minimized, the loss function will yield our optimal β .

`lstsq` returns a list of four things, but for our purposes we're only interested in one of them: the array of the β values for the best-fit line. If you're curious about exactly how `lstsq` works, you can [read the docs](#) for more information, but that isn't necessary to complete this notebook.

```
In [10]: # calculate the least squares solution
lstsq_results = lstsq(X_train, y_train)

beta = lstsq_results[0]

beta
```

```
Out[10]: array([ 1940.25802773,  6390.88660014,    84.39369242, -859.22813236,
                -1298.28862353, -497.56440709])
```

We now have everything we need to make predictions about the total number of riders on a given day. Remember, the formula is:

$$\mu_{y|x_1, x_2, \dots, x_k} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k$$

So, to make a prediction for a given day, we take all the values in the X matrix corresponding to that day, multiply each value by its corresponding β , then add them all together. The @ operator can help us with this matrix multiplication.

For example, here's the first row of our explanatory variable matrix.

```
In [11]: X_train.loc[0, :]
```

```
Out[11]: intercept      1.000000
         temp           0.344167
         is work day     0.000000
         fall            0.000000
         spring          1.000000
         summer          0.000000
         Name: 0, dtype: float64
```

To get the prediction, we use @ to multiply each item in the row by each corresponding item in the β vector and sum them all up. If you've taken linear algebra, you'll recognize this as the *dot product*.

NOTE: the following images are included to illustrate the concept of array and matrix multiplication

```
In [12]: # multiply the arrays to get the prediction
         X_train.loc[0, :] @ beta
```

```
Out[12]: 2841.5016727109587
```

The @ operator can also work on matrices. To get the predictions for *every* row in X, we use exactly the same syntax.

```
In [13]: # take the dot product of every row of training data and beta
         # the resulting array is a predicted number of total riders for each day
         predict_train = X_train @ beta

         # show the first 5 predictions
         predict_train.head()
```

```
Out[13]: 215    5702.953074
         570    5963.912146
         11     1830.241766
         181    5782.839156
         303    4197.553164
         dtype: float64
```

Now we can add our predictions to our original DataFrame.

```
In [14]: # make a new column in our training data with the predicted total riders
         bike_train["predicted total riders"] = predict_train

         # show the first five rows of the DataFrame
         bike_train.head()
```

```

Out[14]:
   instant      date  season  year  month  is holiday  week day \
215      216 2011-08-04   fall  2011      8         no  thursday
570      571 2012-07-24   fall  2012      7         no   tuesday
11       12 2011-01-12  spring  2011      1         no  wednesday
181     182 2011-07-01   fall  2011      7         no   friday
303     304 2011-10-31  winter  2011     10         no   monday

   is work day  weather    temp  felt temp  humidity  windspeed \
215         yes      2  0.710000  0.664788  0.757500  0.197150
570         yes      1  0.750833  0.706450  0.655000  0.211454
11         yes      1  0.172727  0.160473  0.599545  0.304627
181         yes      1  0.722500  0.652162  0.396250  0.102608
303         yes      1  0.340000  0.356063  0.703333  0.106350

   casual riders  registered riders  total riders  predicted total riders
215             799             3777           4576           5702.953074
570            1140             6452           7592           5963.912146
11              25             1137           1162           1830.241766
181            1246             4116           5362           5782.839156
303            362             3307           3669           4197.553164

```

EXERCISE: We also want to make predictions for the test data using the β we found

```

In [15]: # make predictions for the test data using beta and the dot product
predict_test = X_test @ beta

# create a new column in our test data with predicted total riders
bike_test["predicted total riders"] = predict_test

# show the first 5 rows of test data
bike_test.head()

```

```

Out[15]:
   instant      date  season  year  month  is holiday  week day \
431      432 2012-03-07  spring  2012      3         no  wednesday
225      226 2011-08-14   fall  2011      8         no   sunday
491      492 2012-05-06  summer  2012      5         no   sunday
231      232 2011-08-20   fall  2011      8         no  saturday
607      608 2012-08-30   fall  2012      8         no  thursday

   is work day  weather    temp  felt temp  humidity  windspeed \
431         yes      1  0.404167  0.385100  0.513333  0.345779
225         no      2  0.676667  0.624388  0.817500  0.222633
491         no      2  0.562500  0.546737  0.740000  0.149879
231         no      1  0.697500  0.648996  0.674167  0.104467
607         yes      1  0.706667  0.652779  0.590417  0.077117

   casual riders  registered riders  total riders  predicted total riders

```


431	432	4484	4916	3309.348561
225	1338	2482	3820	5405.531958
491	2135	4224	6359	5037.567333
231	1914	3277	5191	5538.673299
607	1172	6541	7713	5681.652249

```
In [16]: # run this cell to check your answer
         check("tests/predict-test.ok")
```

```
Out[16]: <gofer.ok.OKTestsResult at 0x7f8692adf9e8>
```

1.3.3 7c. Evaluating the model

Our model makes predictions, but how good are they? We can start to get a sense of how we did by plotting the predictions versus the actual values on our training data on a scatter plot. If our model predicts perfectly:

- the predicted values will be equal to the actual values
- all points in the scatter plot will fall along a straight line with a slope of 1

To do this, we'll use a Seaborn function called `regplot`. `regplot` creates the scatter plot, then adds the linear regression line of best fit. The translucent band around the line represents the 95% confidence interval for the regression estimate- we'll talk more about confidence intervals in a future notebook.

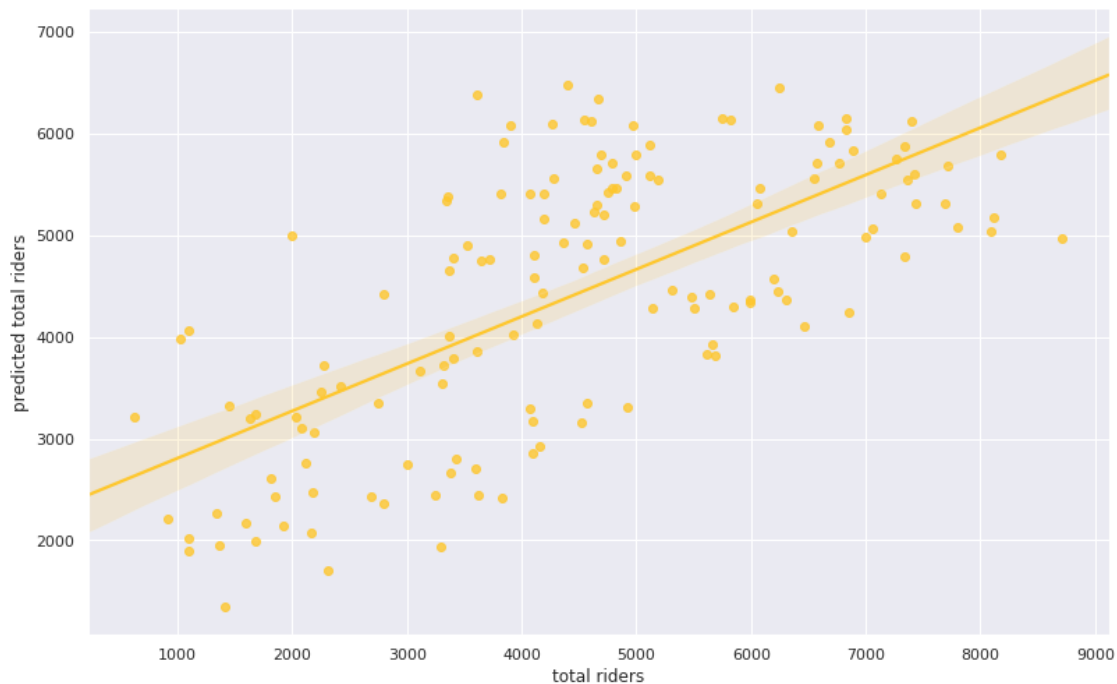
```
In [17]: # make the plots bigger
         sns.set(rc={'figure.figsize':(11.7,8.27)})

         # create the scatter plot and regression line
         sns.regplot(x="total riders", y="predicted total riders", data=bike_train);
```



Here are the test set predictions scattered against the actual total rider counts. Note that we've added an extra color argument to `regplot` to change the color of the dots and line and distinguish the test data from the training data.

```
In [18]: # create the scatter plot and regression line
sns.regplot(x="total riders", y="predicted total riders", data=bike_test, color=(1.0,
```



1.3.4 Root Mean Squared Error

Now it would be nice to have a quantitative measure of how good our model is. An intuitive way to evaluate our model is to look at the *error* for each prediction: the size of the difference between the number of riders we predicted and the actual number of riders. We can get this for each item in our data set by **subtracting the array of predictions from the array of actual values**:

```
In [19]: # get an array of errors
errors = y_train - predict_train

# show the first five items in the error array
errors.head()
```

```
Out[19]: 215    -1126.953074
         570     1628.087854
         11     -668.241766
```

```
181    -420.839156
303    -528.553164
dtype: float64
```

Now we know the error for each individual prediction, but how is the model doing in general? Let's try taking the average of the errors using the `np.average` function:

```
In [20]: # take the average of the errors
np.average(errors)
```

```
Out [20]: -1.9248894715605123e-12
```

The `e-13` is scientific notation: it means that we should take the number to the left of it and multiply it by 10^{-13} (0.0000000000001). The resulting number is very, very close to zero. So, if the average error is almost zero, that would seem to imply that our model is, on average, making near perfect predictions. We know this is untrue thanks to our scatter plot.

What happened?

The problem is that about half of our errors are negative (overestimation) and the other half are positive (underestimation). This is actually a good thing- it means that our model isn't systematically over or underestimating. However, it means that averaging the errors is not a good quantitative measurement of our model, since the negative errors cancel out the positive errors.

In this case, we care about the *magnitude* of the errors: how far each prediction is from the actual value regardless of whether the prediction is over or under. To make all of our errors positive while retaining the magnitude, let's **square all of our errors**. In Python, `**` is the exponential operation; `3 ** 2` tells the computer to multiply 3 by itself 2 times.

```
In [21]: # square the array of errors
sq_error = errors ** 2

# show the first five squared errors
sq_error.head()
```

```
Out [21]: 215    1.270023e+06
570    2.650670e+06
11     4.465471e+05
181    1.771056e+05
303    2.793684e+05
dtype: float64
```

Now that none of our errors are negative, we can safely **take the average**:

```
In [22]: # take the average of the squared errors
mean_sq_error = np.average(sq_error)
mean_sq_error
```

```
Out [22]: 2055728.1341141716
```

The units for the mean squared error are squared riders, which is hard to interpret in real life. Ideally, we would want an average error measure in *riders* so we could say that on average, our model is off by this many riders.

Since mean squared error is in squared riders, we can get a number with a unit of riders by **taking the square root** of our mean squared error:

```
In [23]: # take the square root of the mean squared errors
root_mean_sq_err = np.sqrt(mean_sq_error)
root_mean_sq_err
```

```
Out[23]: 1433.78106212705
```

This number is the **Root Mean Squared Error** (RMSE, sometimes also called the Root Mean Squared Deviation). RMSE is our quantitative measure of the average performance of our model. To summarize the steps:

1. Find the difference between the actual and predicted value for each of our data points (the errors)
2. Square all of the errors
3. Take the average of the squared errors
4. Take the square root of the average squared error

If you like formulas, here it is in a formula:

$$rmse = \sqrt{\frac{\sum_{i=1}^n (y_i - prediction_i)^2}{n}}$$

Next, we want to see what the RMSE is for our test set predictions.

EXERCISE: Calculate the root mean squared error for the test set. Follow each of the ab

Before you run the next cell: would you expect the RMSE for the test set would be higher, lower, or about the same as the RMSE for the training set? Why?

```
In [24]: # calculate the error for each data point
test_errors = y_test - predict_test

# square the errors
test_sq_error = test_errors ** 2

# take the mean of the squared errors
test_mean_sq_error = np.average(test_sq_error)

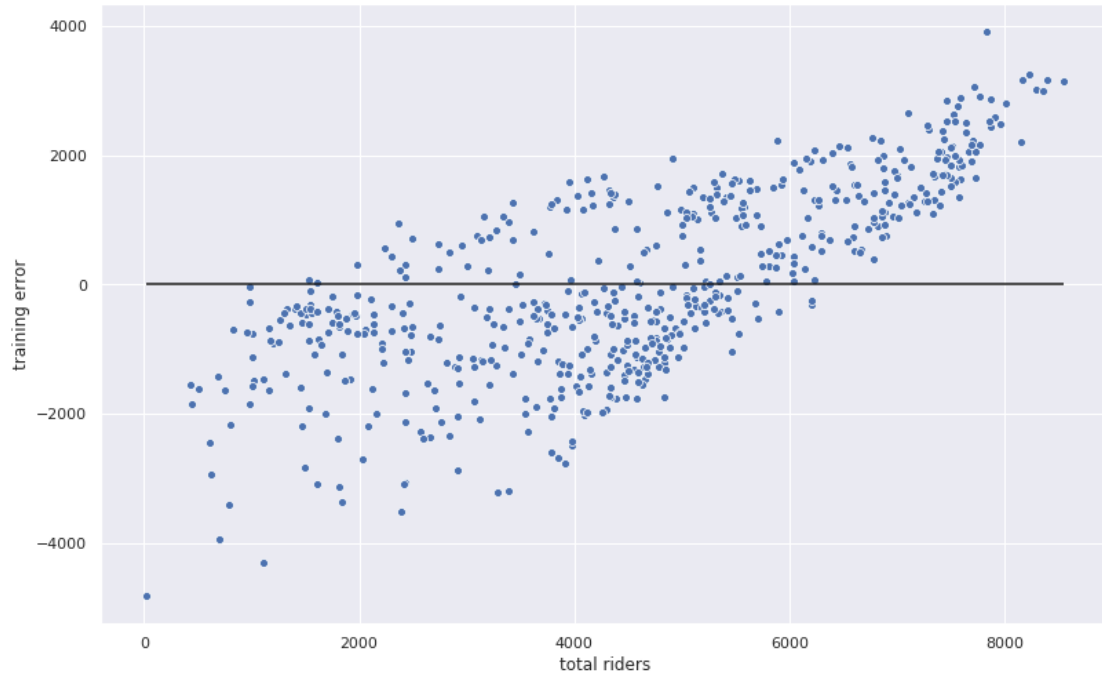
# take the square root of the mean squared errors
test_rmse = np.sqrt(test_mean_sq_error)
test_rmse
```

```
Out[24]: 1401.9698142555396
```

1.3.5 Visualizing Error

We can also visualize our errors compared to the actual values on a scatter plot. The following plots have black horizontal lines at 0 to help see whether our errors are usually over or under zero (that is, whether our model is usually over- or under-estimating), and to how the model makes predictions for different values of our response variable (different numbers of riders).

```
In [25]: # plot the training errors on a scatter plot
bike_train["training error"] = errors
sns.scatterplot(x="total riders", y="training error", data=bike_train);
plt.hlines(0, min(bike_train["total riders"]), max(bike_train["total riders"]));
```



```
In [26]: # plot the test errors on a scatter plot
bike_test["test error"] = y_test - predict_test
sns.scatterplot(x="total riders", y="test error", data=bike_test, color=(1.0, 200/256));
plt.hlines(0, min(bike_test["total riders"]), max(bike_test["total riders"]));
```



QUESTION: Based on the plots and root mean squared error above, how well do you think our model is performing?

ANSWER:

Comparing the plots and the rmse between the training and test data, they are very comparable in performance. The residual plot can be improved by adding even more features

References

- Bike-Sharing data set from University of California Irvine's Machine Learning Repository <https://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset>
- Portions of text and code adapted from Professor Jonathan Marshall's Legal Studies 190 (Data, Prediction, and Law) course materials: [lab 2-22-18, Linear Regression](#) (Author Keeley Takimoto) and [lab 3-22-18, Exploratory Data Analysis](#) (Author Keeley Takimoto)
- "Capital Bikeshare, Washington, DC" header image by [Leeann Caferatta](#) licensed under [CC BY-ND 2.0](#)

In []: